

PLAN DE IMPLEMENTACIÓN ENTORNO DE DESARROLLO BACKEND

PROYECTO: TAG-OK

Tabla de contenidos

Historial de revisiones	3
Introducción	3
Resumen Ejecutivo	4
Alcance de la implementación	4
Elementos del entorno	4
Herramientas y Tecnologías involucradas	5
Preparación del ambiente de desarrollo	6
Configuración del proyecto	6
Configuración de claves y variables de entorno	7
Inicialización de la base de datos	8
Carga de Datos de Calles y Pórticos	9
Verificación de implementación	9
Resolución de problemas comunes	10

Historial de revisiones

Fecha	Versión	Descripción	Autor
20 de Junio de 2026	1.0	Versión inicial del documento	Diego Rodríguez, Paulina Troncoso, Ricardo Sánchez

Introducción

El presente documento describe el procedimiento completo para la implementación y configuración del entorno de desarrollo del backend del proyecto TAG OK. El objetivo es establecer una guía estandarizada que permita a cualquier miembro del equipo preparar su estación de trabajo para ejecutar todos los servicios que componen el ecosistema backend de la plataforma.

El sistema se compone de múltiples servicios integrados: un API Gateway desarrollado con Spring Boot y Spring Cloud Gateway WebFlux que actúa como punto de entrada único en el puerto 8080, un servicio de rutas y tarifas que utiliza PostgreSQL con PostGIS y pgRouting para el cálculo de rutas óptimas, un servicio de historial que persiste en MongoDB, y un broker de eventos Apache Kafka para la comunicación asíncrona entre servicios.

Este plan cubre desde la instalación de los prerequisites base hasta la verificación del sistema completo en funcionamiento, incluyendo la configuración de variables de entorno, la inicialización de las bases de datos y la carga de datos geoespaciales y de pódicos.

Resumen Ejecutivo

Para ejecutar el backend de TAG OK se requiere un entorno compuesto por múltiples servicios coordinados mediante Docker Compose. Este plan describe la implementación en cuatro fases principales.

La primera fase consiste en la preparación del entorno base, que incluye la instalación de Docker Desktop, Java JDK 21 y Maven. La segunda fase abarca la configuración del archivo de variables de entorno .env con las credenciales de Supabase, MongoDB, PostgreSQL, Kafka y Gemini. La tercera fase se enfoca en el levantamiento de los servicios mediante Docker Compose y la verificación de su correcta inicialización. La cuarta fase comprende la carga de datos esenciales: las calles de Santiago mediante la herramienta osm-importer y los pódicos de las autopistas mediante el panel de administración web.

El resultado es una guía paso a paso que permite tener todo el ecosistema backend ejecutándose y listo para recibir peticiones desde la aplicación móvil y el panel web.

Alcance de la implementación

Este documento abarca la configuración del entorno necesario para compilar y ejecutar todos los servicios backend del proyecto TAG OK. El alcance incluye la instalación de Docker Desktop y la verificación de la virtualización, la instalación del JDK 21 y Maven, la configuración del archivo .env con todas las variables de entorno requeridas, el levantamiento de los contenedores mediante Docker Compose, la inicialización del esquema de base de datos en Supabase, la carga de datos de calles mediante la herramienta osm-importer, y la carga de pódicos de autopistas mediante el panel de administración web.

Queda fuera del alcance la configuración del entorno de desarrollo de la aplicación móvil Android y del frontend web, los cuales se tratan en documentos separados.

Elementos del entorno

El entorno de desarrollo backend está compuesto por los siguientes elementos de software y configuraciones. Como sistema operativo base se utiliza Windows 10/11 con soporte para virtualización, macOS o Linux. Docker Desktop es la plataforma de contenedores que orquesta todos los servicios. El JDK 21 y Maven son necesarios para la compilación de los proyectos Java y la herramienta osm-importer.

Los servicios que componen el sistema son el gateway-service como punto de entrada único en el puerto 8080 desarrollado con Spring Boot y Spring Cloud Gateway WebFlux, el routes-service como API de rutas y tarifas que utiliza PostgreSQL con PostGIS y pgRouting, el history-service como API de historial que utiliza MongoDB, y Apache Kafka con Zookeeper como broker de eventos para la comunicación asíncrona entre servicios.

Los servicios externos utilizados son Supabase para la autenticación de usuarios, y Gemini como API de inteligencia artificial. Las bases de datos incluyen PostgreSQL con las extensiones PostGIS y pgRouting para los datos geoespaciales, y MongoDB para el almacenamiento documental del historial.

Herramientas y Tecnologías involucradas

Las herramientas utilizadas para la implementación del entorno backend son Docker Desktop como plataforma de contenedores que permite orquestar todos los servicios mediante un archivo docker-compose.yml, garantizando la consistencia del entorno entre diferentes estaciones de trabajo.

Docker Compose se utiliza para definir y ejecutar los contenedores de PostgreSQL, MongoDB, Kafka, Zookeeper y los servicios propios del proyecto. El JDK 21 proporciona el entorno de ejecución para los servicios desarrollados en Java con Spring Boot. Maven es la herramienta de build y gestión de dependencias para los proyectos Java, incluyendo la herramienta osm-importer.

El API Gateway desarrollado con Spring Cloud Gateway WebFlux sobre Netty actúa como punto de entrada único, enrutando las peticiones a los servicios correspondientes e integrando la autenticación con Supabase. PostgreSQL con las extensiones PostGIS y pgRouting proporciona la capacidad de almacenar datos geoespaciales y ejecutar algoritmos de enrutamiento como pgr_dijkstra. MongoDB ofrece flexibilidad en el esquema para el almacenamiento del historial de cruces. Apache Kafka permite la comunicación asíncrona y desacoplada entre servicios.

Preparación del Ambiente de Desarrollo

El primer paso consiste en instalar Docker Desktop descargando el instalador desde el sitio oficial de Docker. Durante la instalación, es necesario asegurarse de que la virtualización esté habilitada en el sistema operativo. En Windows, esto implica verificar que Hyper-V o WSL 2 estén correctamente configurados. Una vez instalado, se debe iniciar Docker

Desktop y esperar a que el motor de Docker esté completamente operativo antes de continuar.

El segundo paso es instalar el JDK 21 descargándolo desde el sitio oficial de Oracle o utilizando un gestor de paquetes como SDKMAN. Es importante verificar la instalación ejecutando el comando `java -version` en una terminal y confirmando que se muestre la versión 21.

El tercer paso es instalar Maven descargándolo desde el sitio oficial de Apache Maven o utilizando un gestor de paquetes. Se debe verificar la instalación ejecutando el comando `mvn -version` y confirmando que tanto Maven como el JDK 21 sean reconocidos correctamente.

Preparación del ambiente de desarrollo

El primer paso consiste en instalar Android Studio descargando el instalador desde el sitio oficial de Android Developers. Durante la instalación, se debe aceptar la configuración de tipo Standard en el asistente de inicio, lo que descargará automáticamente el JDK 21 y los componentes más recientes del SDK de Android.

Una vez instalado, se debe verificar y configurar el SDK de Android accediendo a la sección de configuración del IDE. En la pestaña de SDK Platforms se debe marcar la casilla para mostrar los detalles del paquete y asegurar que para Android 14.0 esté instalado el Android SDK Platform 34. En la pestaña de SDK Tools se debe verificar la instalación de Android SDK Build-Tools versión 34 o superior, Android Emulator, Android SDK Platform-Tools y el acelerador de emulador correspondiente al procesador.

El tercer paso es crear un dispositivo virtual mediante el Administrador de Dispositivos. Se recomienda seleccionar un hardware como Pixel 7, utilizar una imagen del sistema para API 34 y asignar un nombre descriptivo al dispositivo virtual. Una vez creado, se debe iniciar para verificar su correcto funcionamiento.

Configuración del proyecto

Con los prerequisites instalados, se procede a clonar el repositorio del proyecto desde la terminal. Una vez clonado, se navega hasta la carpeta Producto, que contiene el archivo `docker-compose.yml` principal del sistema.

La estructura del proyecto está organizada en módulos independientes. En la raíz de Producto se encuentra el archivo docker-compose.yml que orquesta todos los servicios. El módulo routes-service contiene la API REST de rutas y tarifas. El módulo history-service contiene la API de historial con MongoDB. El módulo osm-importer contiene la herramienta de importación de datos OSM. La carpeta porticos contiene los archivos JSON con los datos de pódicos de cada autopista. La carpeta supabase contiene el esquema SQL para las tablas de autenticación.

Configuración de claves y variables de entorno

Este paso es crítico para el correcto funcionamiento del sistema. Se debe crear un archivo .env en la carpeta Producto con todas las variables de entorno requeridas por los servicios.

La variable SERVER_PORT define el puerto en el que se expone el API Gateway, siendo el valor predeterminado 8080. Las variables SUPABASE_URL y SUPABASE_ANON_KEY contienen la URL del proyecto y la clave anónima de Supabase para la autenticación. La variable SUPABASE_JWT_ISSUER_URI define el endpoint de emisión de tokens JWT de Supabase.

La variable MONGODB_URI contiene la cadena de conexión a la base de datos MongoDB utilizada por el servicio de historial. Las variables POSTGRES_DB, POSTGRES_USER y POSTGRES_PASSWORD definen las credenciales de conexión a la base de datos PostgreSQL utilizada por el servicio de rutas.

La variable KAFKA_BOOTSTRAP_SERVERS define la dirección del broker de Kafka para la comunicación asíncrona. Las variables GEMINI_API_KEY y GEMINI_MODEL contienen la clave de API y el modelo de Gemini para las funcionalidades de inteligencia artificial.

Es fundamental verificar que ninguna de estas variables contenga valores reales de producción y que el archivo .env esté incluido en el archivo .gitignore para evitar exponer credenciales en el repositorio.

se deja .env.example en el repositorio

```
# Gateway service configuration
SERVER_PORT=8080

# Supabase authentication endpoint
SUPABASE_URL=https://ibafvqmoqeabmziyzfk.supabase.co
```

```
SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJzdXBhYmFzZSIsInJlZiI6ImliYWZ2cWlvcWVhYm16aXl6aWZrIiwicm9sZSI6ImFub24iLCJpYXQiOiOjE3NzY3ODc5NTIsImV4cCI6MjA5MjMzMzk1Mn0.ZYYd0xW69sq1CyT6DqsMj23zFSfedrGaed35AhE-GEs
SUPABASE_JWT_ISSUER_URI=https://ibafvqmoqeabmziyzifk.supabase.co/auth/v1

# History service configuration
# Mongo
MONGODB_URI=mongodb://admin:admin@db-historial:27017/historial_db?authSource=admin

# Route Service
# PostgreSQL
POSTGRES_DB=db_rutas
POSTGRES_USER=admin
POSTGRES_PASSWORD=admin

# Kafka
KAFKA_BOOTSTRAP_SERVERS=kafka-tag:9092

# Gemini
GEMINI_API_KEY=AQ.Ab8RN6IgVEwJEUFbHM7tmdKgpGhguOfgduGDGBx_Xufi3u-8Wg
GEMINI_MODEL=gemini-2.5-flash
```

Inicialización de la base de datos

El sistema requiere la inicialización del esquema de base de datos en Supabase. El esquema SQL se encuentra en el archivo `supabase-schema.sql` dentro de la carpeta Producto. Este script crea las tablas necesarias para la autenticación de usuarios y las funciones de presupuesto mediante edge functions.

Para inicializar el esquema, se debe acceder a la consola de Supabase, navegar al editor SQL y ejecutar el contenido del archivo `supabase-schema.sql`. Esto creará toda la estructura de tablas y funciones requeridas por la aplicación.

Carga de Datos de Calles y Pórticos

El sistema necesita dos conjuntos de datos fundamentales para su funcionamiento: las calles de Santiago y los pórticos de las autopistas concesionadas.

Para la carga de calles se utiliza la herramienta `osm-importer`, que procesa archivos de exportación en formato JSON de OpenStreetMap y los importa a la base de datos PostgreSQL con la estructura requerida por `pgRouting`. El procedimiento consiste en navegar hasta la carpeta `osm-importer` y ejecutar tres comandos secuenciales.

Primero se compila el proyecto con el comando `mvn compile`. Luego se cargan las dependencias con el comando `mvn dependency:copy-dependencies`. Finalmente se ejecuta la herramienta con el comando `java -cp "target/classes;target/dependency/*" com.roony.Main`, que procesa los archivos JSON de calles ubicados en la carpeta `resources/datos-calles` y los importa a la base de datos del servicio de rutas.

Para la carga de pórticos, se utiliza el panel de administración web `routes-ui`. Una vez que todos los servicios estén levantados, se accede al panel y se utiliza la funcionalidad de carga masiva para importar los archivos JSON ubicados en la carpeta `porticos`. El sistema valida automáticamente el formato de cada archivo y solo acepta aquellos que cumplan con la estructura requerida.

Verificación de implementación

Para confirmar que todo el entorno backend está correctamente configurado, se debe proceder con el levantamiento de los servicios. Estando en la carpeta `Producto`, se ejecuta el comando `docker compose up gateway-service -d` para levantar únicamente los servicios backend, o `docker compose up frontend -d` para levantar también el panel de administración web. Docker Compose se encargará de descargar las imágenes necesarias, construir los contenedores y establecer la red interna entre servicios.

Una vez que todos los contenedores estén en ejecución, se verifica el estado con el comando `docker ps`, donde deberían aparecer los contenedores de PostgreSQL, MongoDB, Kafka, Zookeeper y los servicios propios del proyecto.

Se verifica que el API Gateway esté respondiendo accediendo a la documentación de Swagger en la URL `http://localhost:8080/swagger-ui/index.html`. Esta interfaz muestra todos

los endpoints disponibles organizados por servicio, incluyendo los recursos de rutas en `/api/routes` y los recursos de historial en `/api/history`.

Se verifica que el panel de administración web esté accesible en `http://localhost:5173` y que muestre correctamente el mapa interactivo. Se realiza una prueba de autenticación con Supabase para confirmar la correcta configuración de las variables de entorno.

Resolución de problemas comunes

Durante el proceso de configuración pueden presentarse diversos errores. Si Docker Desktop no inicia correctamente, se debe verificar que la virtualización esté habilitada en la BIOS del equipo y que Hyper-V o WSL 2 estén correctamente configurados en Windows.

Si los contenedores no logran comunicarse entre sí, se debe verificar que todos estén en la misma red de Docker y que los nombres de host definidos en el archivo `docker-compose.yml` coincidan con los utilizados en las cadenas de conexión de las variables de entorno.

Si la conexión a Supabase falla con errores de autenticación, se debe verificar que las variables `SUPABASE_URL`, `SUPABASE_ANON_KEY` y `SUPABASE_JWT_ISSUER_URI` estén correctamente configuradas en el archivo `.env` y que el esquema SQL haya sido ejecutado en la consola de Supabase.

Si la carga de calles con `osm-importer` falla, se debe verificar que PostgreSQL con PostGIS y pgRouting esté correctamente inicializado y accesible, que las credenciales de conexión en el archivo `.env` sean correctas, y que los archivos JSON de calles estén presentes en la carpeta `resources/datos-calles` del proyecto `osm-importer`.

Si la carga de pórticos desde el panel web falla, se debe verificar que los archivos JSON tengan el formato requerido por el sistema y que el servicio `routes-service` esté correctamente levantado y accesible desde el panel de administración.

Si Kafka no inicia correctamente, se debe verificar que Zookeeper esté levantado antes que Kafka, ya que Kafka depende de Zookeeper para su funcionamiento. También se debe verificar que la variable `KAFKA_BOOTSTRAP_SERVERS` apunte al host y puerto correctos.